

Project Report  
Real-Estate Office Database  
Deborah Maignan  
CSC 4710  
31 July 2026

## Abstract

This report documents the conceptual, logical, and physical design of a relational database for a small residential real-estate office, along with a working implementation, its operating procedures, and a maintenance plan. The system tracks agents, buyers, sellers, property listings, and completed sales. An entity–relationship model was developed for the domain and mapped to a five-table relational schema enforced through primary and foreign keys. The schema was implemented as an embedded SQLite database accessed from a menu-driven Java console application over JDBC. The application supports full create, read, update, and delete operations for every entity, records property sales, and provides search and reporting queries. This report covers all five phases of the project: conceptual design, logical design, implementation, operation, and maintenance.

## Introduction

Small real-estate offices generally track the same recurring information: which agents work for the office, which buyers and sellers they represent, which properties are currently listed, and which properties have already sold. Managing this information in spreadsheets or paper records can become error-prone as the number of listings grow, particularly when the same person may act as both a buyer's agent and a listing agent on different transactions. This project designs and implements a relational database to support these operations.

This project followed the standard database life cycle used from the project overview: conceptual design, in which the entities and relationships of the domain were identified independent of any particular database product; logical design, in which the conceptual model was mapped to a relational schema; implementation, in which the schema was created and a manufactured client application was built against it; operation, describing how the system is compiled, run, and used day to day; and lastly, maintenance, describing how the system is kept correct and available over time.

## Conceptual Design

The domain was modeled around five entities: Agent, Buyer, Seller, Property, and Sale. An Agent represents an employee of the real-estate office who may list properties on behalf of sellers and may represent buyers or sellers in a transaction. A Buyer and a Seller are external parties identified by the standard contact information. A Property represents a residential listing, described by its address, city, school district, bedroom and bathroom counts, asking price, and

whether it has a pool. A Sale records the completed transfer of a specific property, including the sale price, sale date, the buyer, the seller, and the selling agent.

Four relationships connect these entities. An Agent lists zero or more Property records (one-to-many); a Seller owns zero or more Property records (one-to-many); a Property is sold through at most one Sale (one-to-one, and only once it has been sold); and a Buyer, a Seller, and up to two Agents each participate in a given Sale. The last relationship is the reason Sale is modeled as its own entity rather than as extra columns on Property: a transaction naturally involves two distinct agent roles — the agent who listed the property and, separately, the agent who represented the buyer — and collapsing that into a single table would not represent both roles cleanly.

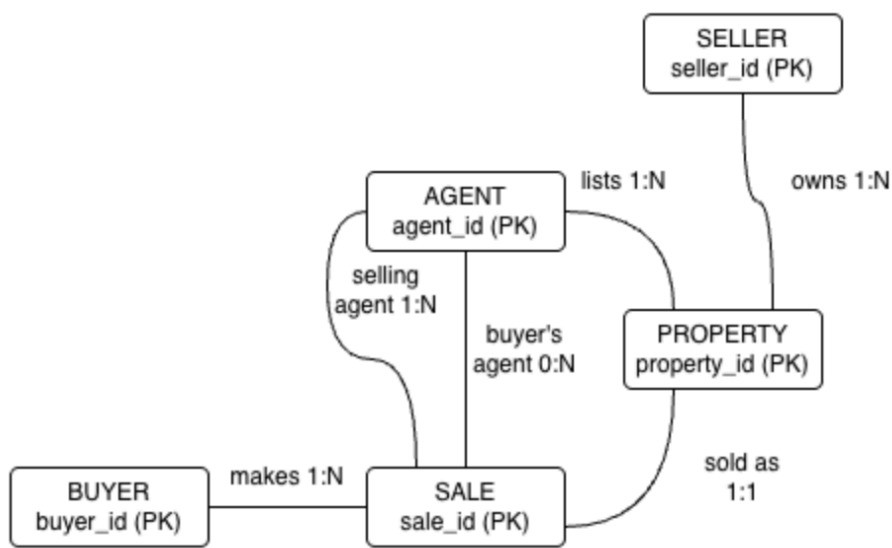


Figure 1. Entity-relationship diagram

### Logical Design

Each entity from the conceptual model maps to exactly one relational table, and each relationship is realized as a foreign key rather than as a separate table, since none of the relationships in this domain are many-to-many. Primary keys are created as surrogate integer identifiers (agent\_id, buyer\_id, seller\_id, property\_id, sale\_id) rather than natural keys such as phone numbers, both because natural keys can change and because a single-column integer key keeps every foreign key reference simple.

The property table carries two foreign keys back to the entities that participate in a listing: `seller_id`, identifying who owns the property, and `listing_agent`, identifying which agent listed it. The sale table carries five foreign keys: `property_id`, `buyer_id`, and `seller_id` identify the three parties to the transaction beyond the office itself, and `selling_agent` and `buyers_agent` identify the agents involved, with `buyers_agent` left nullable to represent an unrepresented buyer.

This schema is in third normal form for the scope of the project: every non-key attribute in each table depends on the whole primary key and on nothing but the primary key. The one design choice worth noting is that `school_district` and `city` are stored directly on the property as text rather than normalized into a separate lookup table since this is for a simple real estate office.

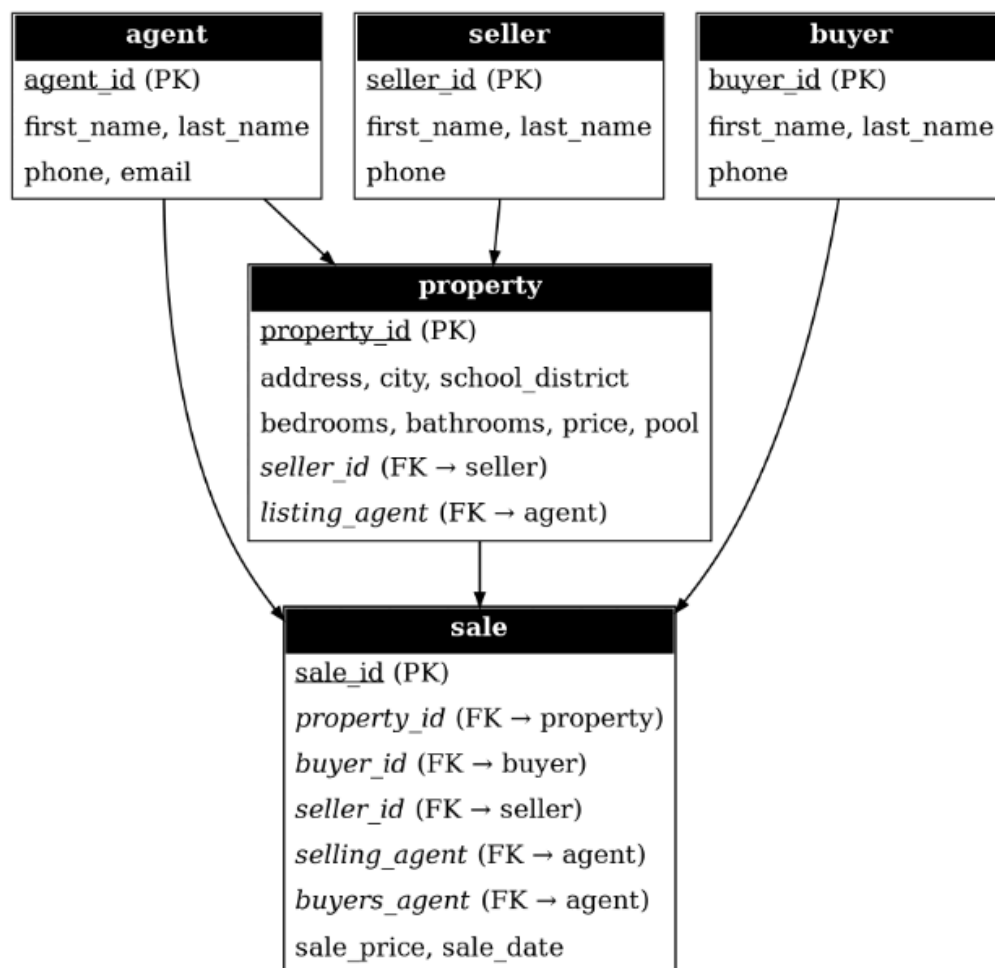


Figure. 2. Relational schema

## Implementation

The schema was implemented on SQLite, an embedded, file-based database engine that requires no separate server process. SQLite was chosen over a client–server engine because the target deployment is a single office workstation running a single-user console application; an embedded database removes the need to install, configure, and keep running a separate database server, and the entire database is a single portable file (realestate.db).

The client is a Java console application, `RealEstateApp.java`, that connects to SQLite through the JDBC API using the `sqlite-jdbc` driver. On startup, the application executes a `CREATE TABLE IF NOT EXISTS` statement for each of the five tables, so the schema is created automatically on first run and left untouched on subsequent runs. All statements that accept user-supplied values use `PreparedStatement` with bound parameters rather than string concatenation, which both simplifies type handling and prevents SQL injection. A single generic helper executes every parameterized update statement:

```
private static void executeUpdate(
    String sql, Object... params) {
    try (Connection conn =
        DriverManager.getConnection(DB_URL);
        PreparedStatement pstmt =
            conn.prepareStatement(sql)) {
        for (int i = 0; i < params.length; i++)
            pstmt.setObject(i + 1, params[i]);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.out.println(e.getMessage());
    }
}
```

Around this data-access layer, the application presents a text menu with options grouped by entity — list, add, update, and delete for Property, Agent, Buyer, and Seller — plus options to record a sale, list all sales, and run four search and reporting queries (by city, by maximum price, by school district, and average sale price / most expensive property).

## Operation

The application is compiled and run from the command line with the JDBC driver on the classpath:

```
javac -cp sqlite-jdbc.jar RealEstateApp.java
java -cp .:sqlite-jdbc.jar RealEstateApp
```

The first run creates realestate.db in the working directory; later runs reuse that same file, so data persists across sessions. A typical session selects a numbered option from the menu, is prompted for the relevant fields on separate lines, and receives immediate confirmation of how many rows were affected. For example, recording a sale prompts for the sale, property, buyer, and seller identifiers, the selling and buyer's agent identifiers, the sale price, and the sale date, and inserts one row into the sale table; searching by maximum price prompts for a single dollar amount and lists every property whose price is at or below it.

## Maintenance

### ***A. Backup and Version Control***

Because the entire database is one file, backup is a matter of periodically copying realestate.db to a separate location; the application source (RealEstateApp.java) and the schema definitions are kept under version control separately from the data file itself.

### ***B. Schema Evolution***

Using CREATE TABLE IF NOT EXISTS makes re-running the initialization step idempotent, so the application can be safely restarted without disturbing existing data. Any future column additions would be handled with an explicit ALTER TABLE migration rather than by editing the CREATE TABLE statement in place, so that existing installations are updated rather than left behind.

### ***C. Data Integrity and Testing***

SQLite does not enforce foreign-key constraints unless PRAGMA foreign\_keys = ON is issued for each connection; enabling that pragma is recommended future work so that the referential-integrity rules shown in Figure 2 are enforced by the database itself rather than only

by application logic. After any change to the schema or the data-access code, the standard regression check is to re-run the full menu — add, list, update, delete, and search for each entity, plus record a sale — and confirm the data still persists correctly between runs.

#### ***D. Future Work***

The current design supports a single user at a time. Planned extensions include migrating from SQLite to a client–server engine such as PostgreSQL mentioned in the project to support multiple concurrent users, adding stricter input validation on the console prompts, and extending the reporting queries beyond the four currently supported.

#### **Conclusion**

This project carried a real-estate office database through all five phases of the database life cycle mentioned in the project overview. The conceptual model identified five entities and the relationships among them; the logical design mapped that model to a five-table relational schema with explicit primary and foreign keys; the implementation realized that schema in SQLite behind a menu-driven Java/JDBC console application; the operation phase documented how to build, run, and use that application; and the maintenance phase established a backup, schema-evolution, and testing plan along with concrete next steps toward a multi-user deployment.

#### **References**

R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA: Pearson, 2017.

SQLite Consortium, “SQLite Documentation,” [sqlite.org](https://www.sqlite.org/docs.html). [Online]. Available: <https://www.sqlite.org/docs.html>

Xerial Project, “sqlite-jdbc: SQLite JDBC Driver,” GitHub repository. [Online]. Available: <https://github.com/xerial/sqlite-jdbc>

Oracle Corporation, “JDBC Database Access,” Java SE Documentation. [Online]. Available: <https://docs.oracle.com/javase/tutorial/jdbc/>